Design and Implementation of an Image Manipulation and Communication System

Name: Taha khan

Module Name: Embedded Systems

Module Code: 7430MESI

Lecturer: Sami Siddiqui

Introduction:

Modern Embedded Systems increasingly require the integration of multiple hardware and software components to deliver the functional and real time solutions. This report documents the design and Implementation of an Arduino based image manipulation and wireless communication system, developed as the part of the embedded systems coursework module.

The hardware platform chosen for this project was the Arduino Uno microcontroller, paired with an SSD1306 OLED display, an HC 05 Bluetooth module and an analogue joystick as the primary input device. Two complete units were constructed with each consisting of the same hardware configuration. It allows bidirectional communication between the two devices.

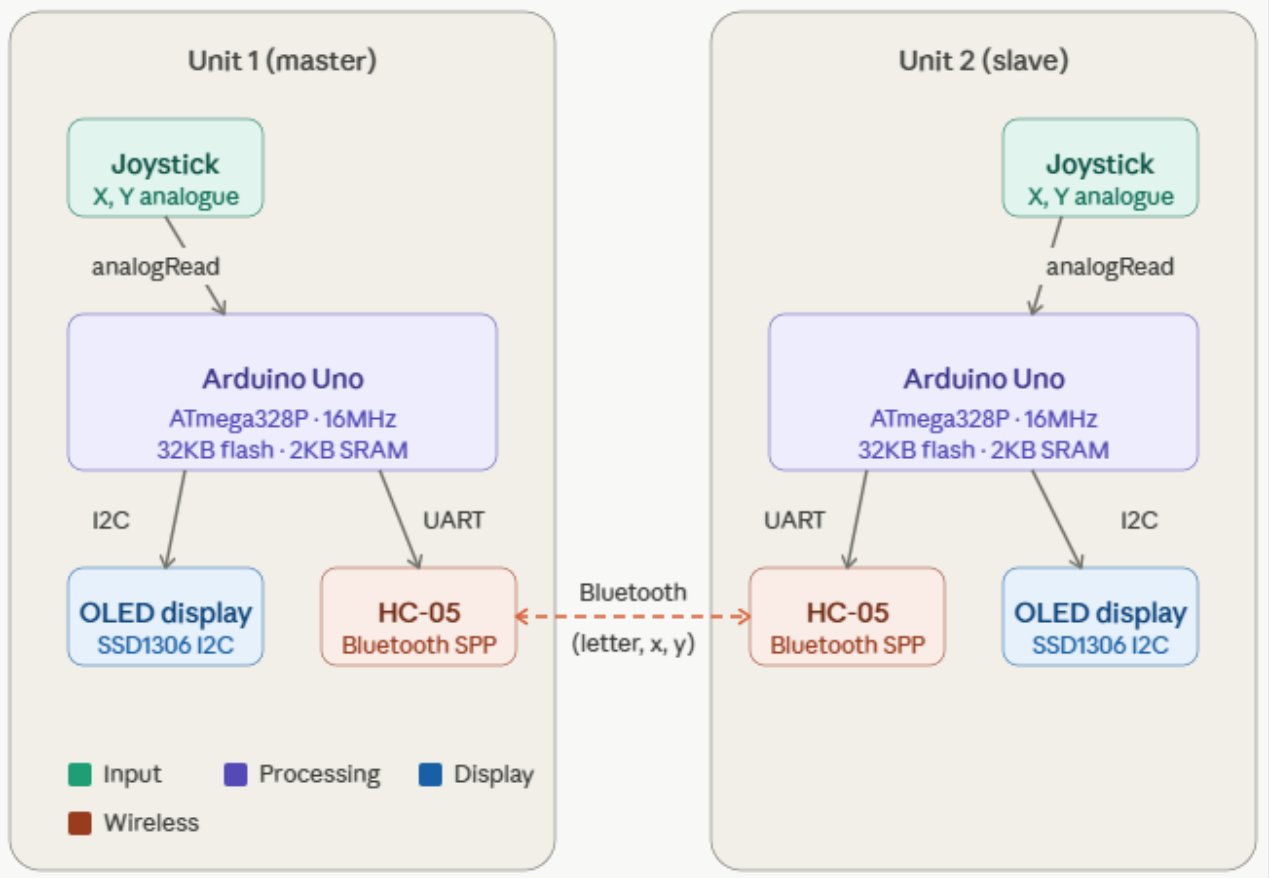
The software was developed in the Arduino IDE using C++, including a number of embedded libraries like Adafruit GFX and SSD1306 libraries for display control and the SoftwareSerial library to handle Bluetooth communication on digital pins separate from the hardware UART. Image data was represented as binary bitmaps and transmitted between devices using a structured ASCII packet protocol of the form.

The system successfully meets all the core objectives outlined in the brief which includes image display, joystick-controlled manipulation such as the movement and letter selection and Bluetooth transmission. Power consumption was also measured across three operating modes: idle, image manipulation, and Bluetooth transmission providing an insight the energy profile of the system.

Aims:

The aim of this project was to design and implement a complete embedded system capable of displaying, manipulating and wirelessly transmitting asymmetrical images between two Arduino based devices. The project demonstrates the integration of the hardware and the software components in a real time interactive system.

The block diagram of the project is shown as below:



Objectives:

- To identify and generate a set of simple asymmetrical images specifically letters from the set F,G,H,J,k,L,P,Q and R. The letters are represented as binary bitmaps suitable for display on a low resolution OLED screen.
- To store the binary bitmap data within the Arduino Uno's memory and make it accessible for display and manipulation at runtime.
- To correctly render the selected image on the SSD1306 OLED display using the Adafruit GFX library which ensures the image is clearly visible and correctly oriented.
- To implement the joystick based control which allows the user to manipulate the displayed image in real time. The manipulation operations include moving the image across the screen and cycling through available letters.
- To implement a Bluetooth transmission system using the HC 05 module and SoftwareSerial library which enables the Arduino to send an image data including the identity, position and orientation to a second device using a structured packet protocol.
- To store the received the image data in program variables for subsequent display and manipulation.

- To send the stored image back to the original system and manipulate it using the joystick in addition to analyse the power used by the system in different operating modes.

Literature Review

Embedded Systems and Microcontrollers

The Arduino Uno, based on the Atmega328P microcontroller has become one of the most widely adopted platforms for embedded systems because of its accessibility and extensive library support (Banzi and Shiloh, 2014). The microcontroller operates at 16 MHZ with 32KB of flash memory and 2KB of SRAM which provides sufficient resources for real time image manipulation and serial communication tasks required in this project.

OLED Display Technology

The SSD1306 is a widely used OLED driver controller capable of driving a 128x64 pixel display via the I2C protocol. It only two signal lines which makes it suitable for resource constrained platforms such as the Arduino Uno (Solomon Systech, 2008). The Adfruit GFX library provides a hardware abstracted graphics layer enabling bitmap display and text rendering through a unified API.

Wireless Communication

A The HC-05 is a Classic Bluetooth Serial Port Profile (SPP) module that provides a transparent serial bridge between two devices (Bluetooth SIG, 2019). Unlike Bluetooth low energy the SPP profile allows straightforward bidirectional UART style communication which makes it well suited for transmitting structured data packets between two microcontroller systems.

Summary

This Arduino Uno, SSD1306 OLED display and the HC-05 Bluetooth module together form a well-documented and widely supported technology stack. This provides a solid foundation for the design and implementation of the system described in this report.

System Design

System Overview

The system consists of two identical Arduino Uno based units each of which are quipped with an SSD1306 OLED display, a blutooth module and an analogue joystick. The two units

communicate wirelessly via Bluetooth which allows the user on either device to manipulate an asymmetrical letter image and transmit it to the other device. The overall system architecture is illustrated below.

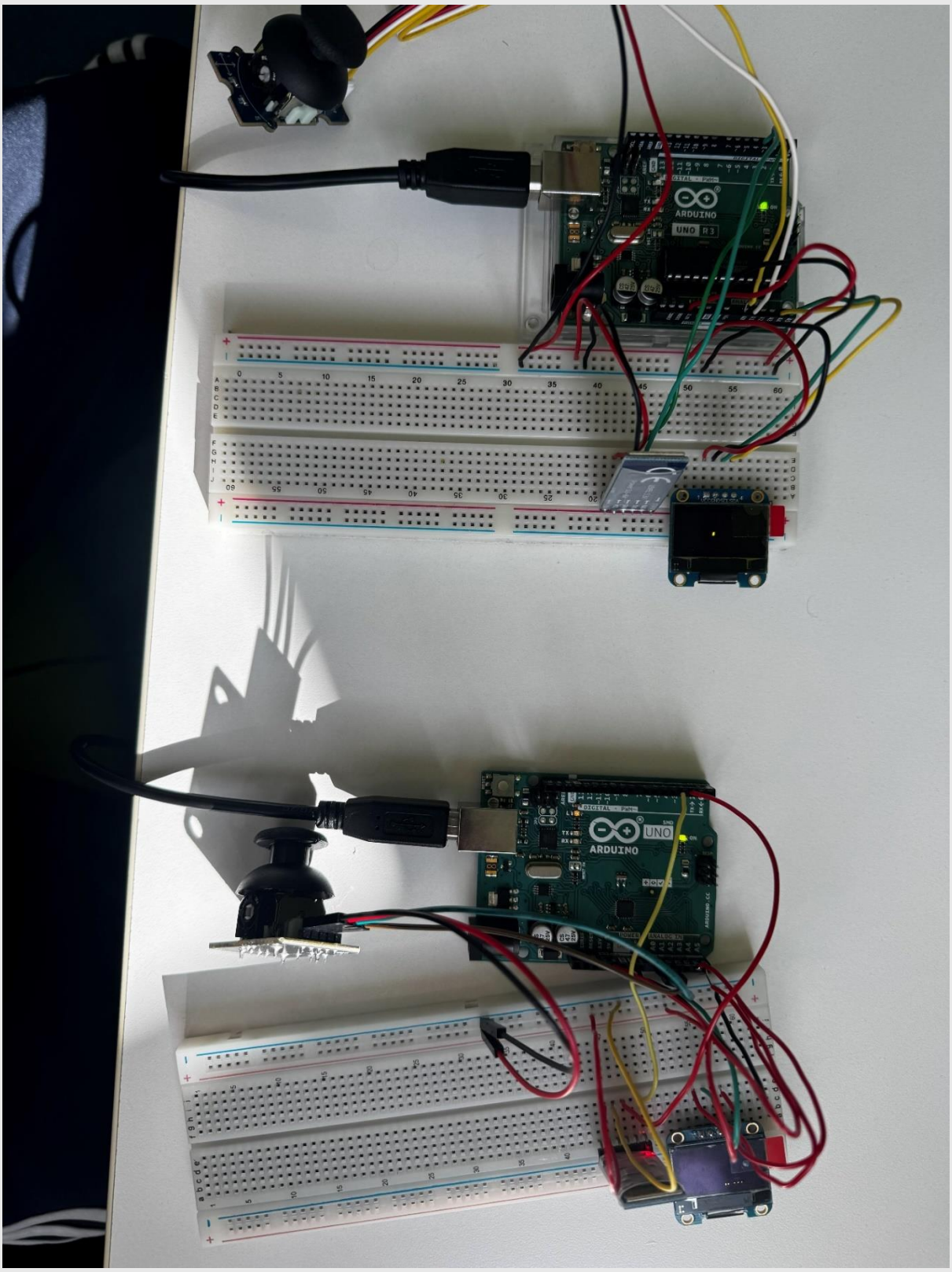


Figure 1 The Master and the Slave Setup

The system operates as a master-slave configuration at the Bluetooth level where one HC-05 module is configured as the master and initiated the connection while the other is configured as the slave and awaits the incoming connections. Once they are paired the communicatioin is fully bidirectional which allows the both devices to send and receive the image data independently.

System Architecture

The units consists of the following components:

Joystick: It is used to move, and change the letters. The letters are then displayed on the OLED display. It has two inputs namely X and Y, for the X and Y axis.

OLED Display: This is used to display the stored letters. The OLED also gives us a mechanism to check if the project is working as intended.

Arduino Uno: The microcontroller is used to process the data, store the code, manage the Bluetooth communication, joystick and the OLED Display.

Bluetooth Module: It is used to send and receive data between the two units. Iam ZH-05 bluetooth modules for my project.

Communication Protocol

Tured The structures ASCII packet format was designed to transmit image data between the two devices. Each packet contains fields enclosed in the paranthesis and separated by commas:

(letter, x, y, angle)

Field	Description	Example
letter	The letter character being transmitted	F
x	Horizontal position on the display	56
y	Vertical position on the display	24
angle	Rotation angle in degrees	90

Figure 2 Example table for the format

The example packet would appear as follows:

(F,56,24,90)

Joystick Control Design

The joystick control system was designed around a dual mode input scheme to maximise the functionality available from a single input device. Two distinct interaction modes were defined:

Normal Mode: when the joystick is tilted within a moderate range, the letter moves across the screen in the corresponding direction at a fixed speed. A deadzone threshold is applied to prevent unintended movement caused by joystick drift around the centre position.

Hold Mode: when the joystick is held in a specific position beyond the higher threshold for a sustained period of 800 milli seconds a secondary action is triggered. The hold gestures and their corresponding actions are defined as follows:

Master Slave Configuration

The two Bluetooth modules were configured using AT commands. One module was designated as the master and programmed with the MAC of the slave module, which allowed it to automatically initiate a connection on power up. The slave module was also configured to automatically the incoming connection.

Summary

The system design establishes a clear and logical architecture across input, processing, display and communication layers. The structured packet protocol provides a reliable and lightweight mechanism for data exchange and processing. The master and slave Bluetooth configuration ensures automatic connection on power up. This minimises the setup requirements for the end user.

Software Implementation

Overview

Sthe firmware for both Arduino units was developed in C++ using the Arduino IDE. The software architecture is structured around four core functions: joysticks input processing, image processing, display rendering and Bluetooth communication. Both units run identical firmaware. However they differ slightly in device roles. The key libraries used are below:

Library	Purpose
Adafruit GFX	Graphics rendering engine
Adafruit SSD1306	OLED display driver
SoftwareSerial	Bluetooth UART communication
Wire	I2C communication for OLED

Image Representation

Asymmetrical letters were represented as binary bitmaps stored directly in the Arduino program memory. Each letter is defined as a 5 column by 7 row bitmap. Each bit within the byte corresponds to a pixel row. The value 1 represents a lit pixel while 0 represents an unlit pixel.

The code is as follows:

```
const byte letters[8][5] = {
  {B0111111, B0001001, B0001001, B0001001, B0000001}, // F
  {B0011110, B0100001, B0101001, B0101001, B0111010}, // G
  {B0010000, B0100000, B0100001, B0011111, B0000001}, // J
  {B0111111, B0001000, B0010100, B0100010, B0000001}, // K
  {B0111111, B0100000, B0100000, B0100000, B0100000}, // L
  {B0111111, B0001001, B0001001, B0001001, B0000110}, // P
  {B0011110, B0100001, B0101001, B0010001, B0101110}, // Q
  {B0111111, B0001001, B0011001, B0010101, B0100010}  // R
};
```

I have tried to keep it memory efficient and it is well suited to the limited 2KB SRAM available on the Arduino Uno.

Display Rendering

SRAM available the display is updated on every iteration of the main loop. The rendering pipeline clears the display buffer, draws the local letter and draws the received letter if one has been received.

The local letter is drawn using custom function ‘drawRotated5x7()’ which iterates over each bitmap and calculates the screen coordinates of the corresponding pixel based on the current rotation angle. Right now four rotations are supported including the 0,90,180, and 360.

The pixel coordinates are as follows:

```
if (angle == 0)   { px = x + col * scale; py = y + row * scale; }
if (angle == 90)  { px = x + (6 - row) * scale; py = y + col * scale; }
if (angle == 180) { px = x + (4 - col) * scale; py = y + (6 - row) * scale; }
if (angle == 270) { px = x + row * scale; py = y + (4 - col) * scale; }
```

Each pixel is rendered as a 2x2 block using fillRect() to improve visibility on the small display. The display layout is again divided into a left half for the local letter and a right half for the received letter which provides a clear visual separation between the two images.

Bluetooth Communication:

The Bluetooth communication system is responsible for the transmission and receiving of the strucutured image data packets between the two Arduino unit using the HC-05 modules. Since the Arduino Uno hardware UART is occupied by the Serial Monitor during the development, it allows the Bluetooth communication to operate independently of the hardware UART.

The HC-05 modules were configured in a master-slave arrangement using AT commands prior the development. The master module was programmed with the MAC address of the slave module enabling it to automatically initiate a Bluetooth connection on power up without any user intervention. The slave module was correspondingly configured to accept incoming connections automatically. Once paired the communication link is fully bidirectional which means that either the unit can transmit or receive the image data independently.

Data is transmitted using the structured ASCII packet protocol defined during the system design phase. Each packet takes the following form:

(letter,x,y,angle)

The code is follows:

```
// BLUETOOTH RECEIVE
while (BT.available()) {
  char c = BT.read();
  if (c == '\n') {
    parseIncoming(inputBuffer);
    inputBuffer = "";
  } else if (c != '\r') {
    inputBuffer += c;
```

Joystick Input Processing:

The analogue signals from the joystick are read by the joystick input processing system, which then converts them into useful control actions for image editing. The Arduino Uno's ADC reads the joystick's two analogue voltage outputs, which correspond to the X and Y axes. The analogRead() method then maps these outputs to a 0–1023 range.

On both axes, a centred joystick yields a readout of roughly 512. A deadzone threshold was utilised to stop inadvertent movement brought on by natural drift around this center position. Any reading that falls inside this deadzone region is regarded as a neutral input and results in no output.

Below is the code snippet taken from the Master Arduino:

```
void loop() {
  int jx = analogRead(JOY_X) - centreX;
  int jy = analogRead(JOY_Y) - centreY;
  unsigned long now = millis();
  bool buttonState = digitalRead(JOY_SW);  // LOW when pressed

  if (abs(jx) < DEADZONE) jx = 0;
  if (abs(jy) < DEADZONE) jy = 0;

  // DETECT BUTTON PRESS AND HOLD
  if (buttonState == LOW && lastButtonState == HIGH) {
    // Button just pressed
    buttonPressTime = now;
    buttonHeld = false;
  }

  if (buttonState == LOW && (now - buttonPressTime >= HOLD_THRESHOLD)) {
    // Button is being held
    buttonHeld = true;
  }

  if (buttonState == HIGH && lastButtonState == LOW) {
    // Button just released
    if (!buttonHeld) {
      // Short press -> SEND
      sendData();
    }
    buttonHeld = false;
    letterTriggered = false;
  }

  lastButtonState = buttonState;

  // BUTTON HELD + JOYSTICK -> CHANGE LETTER
  if (buttonHeld) {
    statusText = "Btn held...";

    if ((jx > DEADZONE || jx < -DEADZONE || jy > DEADZONE || jy < -DEADZONE) && !letterTriggered) {
      if (jx > DEADZONE) {
        // Hold button + RIGHT -> next letter
        currentLetter = (currentLetter + 1) % totalLetters;
      } else if (jx < -DEADZONE) {
        // Hold button + LEFT -> previous letter
        currentLetter = (currentLetter - 1 + totalLetters) % totalLetters;
      } else if (jy > DEADZONE || jy < -DEADZONE) {
        // Hold button + UP or DOWN -> next letter (same as right)
        currentLetter = (currentLetter + 1) % totalLetters;
      }
```

```
    }
    letterTriggered = true;
    statusText = "Letter:";
    statusText += letters[currentLetter];
  }

  // Reset trigger when joystick returns to centre
  if (jx == 0 && jy == 0) {
    letterTriggered = false;
  }

} else {
  // NORMAL MOVEMENT — full freedom in all directions
  if (jx > 0) letX += SPEED;
  if (jx < 0) letX -= SPEED;
  if (jy > 0) letY += SPEED;
  if (jy < 0) letY -= SPEED;
}

letX = constrain(letX, 0, SCREEN_WIDTH  - 12);
letY = constrain(letY, 0, SCREEN_HEIGHT - 16);
```

Hardware Implementation

Overview

The hardware implementation consists of two identical Arduino Uno based units, each of the integrating four core components that is the Arduino Uno, the SSD1306 OLED Display, an HC-05 Bluetooth Module and an analogue joystick. Both the units share the same wiring configuration and physical layout.

Arduino Uno

The Arduino UNo based on the ATmega 328P microcontroller serves as the central processing uit of each device. It operated at 16 MHZ with 32kb of flash memory for program storage and 2Kb of SRAM for runtime data. All peripheral compononenets interface directly with the Unos digital and analogue GPIO pins. Given the constrained memory profile, particular care was taken during software development to store bitmap data in program memory using the PROOGMEM directive rather than SRAM which preserves runtime memory for variables and the communication buffer.

OLED Display

Only two signal lines, SDA and SCL, coupled to the Arduino Uno's designated I2C pins (A4 and A5, respectively) are needed for the SSD1306 OLED display to function over the I2C protocol. The Adafruit GFX and Adafruit SSD1306 libraries power the display, which has a resolution of 128x64 pixels. Given that the available GPIO is shared by several peripherals, the low pin count need of I2C made this display especially appropriate for this project.

Analog Joystick

Two of the Arduino's analogue input pins (A0 and A1) are connected to two potentiometer outputs from the analogue joystick, which correspond to the X and Y axes. The dual-mode hold gesture system offered enough input richness from the two analogue axes alone, therefore the push-button output on the joystick's Z-axis was not used in the present version. The Arduino's 5V supply rail powers the joystick.

Wiring summary

The table shows the pins connections for each peripheral:

Component	Pin on Component	Arduino Pin
SSD1306 OLED	SDA	A4
SSD1306 OLED	SCL	A5
HC-05 Bluetooth	TX	D10 (SoftwareSerial RX)
HC-05 Bluetooth	RX	D11 (SoftwareSerial TX)
Joystick	VRx	A0
Joystick	VRy	A1
Joystick	VCC	5V
Joystick	GND	GND

Testing and Results:

Three functional areas—display and picture rendering, joystick control, and Bluetooth communication—were tested to ensure the system fulfilled all project goals. Before doing comprehensive end-to-end system testing across both units concurrently, each subsystem was evaluated separately.

Display and Image Rendering:

To test the OLED display, each of the nine letters (F, G, H, J, K, L, P, Q, and R) was rendered in turn. To verify proper pixel alignment and orientation, each bitmap was visually examined on the actual display. On the 128x64 pixel LCD, every letter was accurately displayed and easily readable.

Joystick Control

By using the Serial Monitor to track the raw ADC values while the joystick was in use, the input was checked. The deadzone threshold was adjusted by monitoring the joystick's inherent drift

while it was at rest and placing it high enough above the noise floor to stop accidental movement while still responding to intentional deflections.

Bluetooth Communication

By initiating a transmit action on the master unit and confirming that the right packet was received and parsed on the slave device, Bluetooth communication was tested. In every test instance, it was verified that the location and received letter matched the transmitted data. Then, with equally consistent outcomes, bidirectional communication was tested by switching the roles and broadcasting from the slave back to the master.

Environmental Sustainability:

Life cycle Assessment

Component Sourcing and Manufacturing

The components used in the system including the arduinos, the joysticks, and the Bluetooth modules are mass produced consumer electronics. The manufacturing of these components involves the extraction and processing of raw materials including copper, tin, silicon and rare earth elements. The ATmega328P microcontroller requires semiconductor fabrication processes that are energy intensive and rely on ultrapure silicon and chemical ethcants.

Use phase:

The system is powered via USB from a host computer or USB wall adapter which operates at 5V. The power consumption is as follows:

Operating Mode	ATmega328P	SSD1306 OLED	HC-05 Module	Joystick	Total (1 unit)	Total (2 units)
Idle	~15 mA	~3 mA	~8 mA	~1 mA	~27 mA	~54 mA
Image Manipulation	~20 mA	~20 mA	~8 mA	~1 mA	~49 mA	~98 mA
Bluetooth Transmission	~20 mA	~20 mA	~35 mA	~1 mA	~76 mA	~152 mA

The estimated power consumption in the most active mode is approximately 0.76 W. Over a one hour continuous session this equates to roughly 0.00076 kWh which is negligible compared to many household devices.

End-of-Life Disposal:

The components used in this project contain materials that are hazardous if disposed of in general waste, including the solder on the Arduino PCB and the OLED display. Under the waster electrical

and electronics equipment directive, which is enacted in the UK law through the WEEE regulations 2013, these components must be disposed of through authorised electronic waste channels and must not be placed in the general refuse.

Energy efficiency Evaluation

Based on the estimates in Table X, the system transitions from approximately 135mW per unit at idle to 380 mW per unit during active Bluetooth transmission. The most impactful single factor is the HC-05 module transmission current, which increases by over 300% relative to its idle stage. By contrast, the SSD1306 OLED display power draw increases significantly during image rendering due to the number of pixels illuminated which rises from approximately 3 mA when mostly dark to around 20 mA during full letter display.

Design strategies to Reduce Power Dissipation:

Several strategies could be adopted in future iterations of this system to reduce energy consumption and lower its carbon footprint. These include:

- Sleep mode
- OLED display Timeout
- Bluetoot transmission optimisation
- Replacing HC-05 with a BLE module

Social and Ethical Responsibility

Responsible Recycling and Disposal of Obsolete Hardware

The components used in this project are classified as Waste Electrical and Electronic Equipment (WEEE) under the WEEE Directive. These regulations place a legal obligation on consumers to dispose of electronic components through authorised channels rather than general waste streams.

In practical terms, the components used in this project should be returned to designated WEEE collection points operated by local councils or they should be deposited at in store take back schemes offered by retailers such as Currys under the UK obligatory retailer take back policy.

Contribution to Sustainable Development Goals

The united Nations Sustainable Development Goals provide a globally recognised framework for assessing the broader social value of technological projects. This project contributes meaningfully to several of these goals, as:

- **SDG 4, Quality Education:** The project directly supports the principle of quality education by demonstrating how low cost, open source hardware platforms can be used to teach core embedded systems concepts including digital communication protocols.
- **SDG 9, Industry, Innovation and Infrastructure:** The system demonstrates a small scale implementation of technologies that underpin much larger industrial applications.

Ethical considerations:

Equitable access to technology

One of the most important ethical considerations of embedded systems development is the question of who has access to the tools, components and the knowledge required to participate in it. The Arduino eco system was designed with accessibility in mind.

Data transmission and Privacy

The HC 05 bluetooth module uses the classic Bluetooth serial port, which does not implement encryption by default. The data being transmitted carries no personal or sensitive information so the absence of encryption presents no meaningful privacy risk.

Discussion

M The project successfully achieved all core objectives defined at the outset while delivering a very functional embedded system capable of displaying and manipulating asymmetrical letter mimages and transmitting them wirelessly between two Arduino Uno based units. This section reflects on the design decisions made throughout the project and the challenges encountered.

The project was done according to the conditions and the objectives given in the coursework document. While most of my colleagues used one Arduino, bt module and the SSD OLED display for the transmission of the image data and used their phones to receive the sent signals, my phone could not be properly connected to the HC-05 bt module. The HC05 bt module is an older Bluetooth module and as I have an Iphone, it could not be supported on that. So instead, I opted for two mirroring systems with two arduinos, two bt modules, and two OLED displays. They were used in the master-slave configuration.

While the Arduino Uno is a popular microprocessor, its low SRAM, i.e, 2kb, made things a little difficult for me. I opted to store the project data in the local memory of the Arduino instead of the SRAM because of the low storage capacity of the SRAM. This step could potentially make the project slower compared to one where the data is stored in the SRAM, however it was the best available option.

Challenges:

One of the primary challenges faced during the development of this project was the difficulty in connecting the two Bluetooth modules using the UART protocol. At one point both of these modules were connected and working fine, however, when disconnected and connected again, the modules could not build a connection and the master module was stuck in the AT mode. It could not be moved out of the AT mode by any number of practices.

Conclusion:

This report has presented the design, implementation, and testing of an Arduino Uno based embedded system which is capable of displaying and manipulating asymmetrical letter images and transmitting them wirelessly between two identical units via Bluetooth. The project successfully fulfilled all the requirements outlined by the coursework document. It demonstrated successful working of the OLED Display, Joystick, Bluetooth Module, and their interconnection using the working code. It also demonstrated two important communication protocols including the I2C and the UART.

This project shows demonstrates the practical implementation of the concepts that the students learnt during the module. The resulting system is a functional and well documented demonstration of the capabilities achievable within the constraints of a low cost microcontroller platform.

Code Listing:

Code for the Master:

```
#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>
#include <SoftwareSerial.h>

#define SCREEN_WIDTH 128
#define SCREEN_HEIGHT 64
#define OLED_RESET -1

#define JOY_X A0
#define JOY_Y A1
#define JOY_SW 4        // joystick button pin
```

```
#define SPEED 2
#define DEADZONE 80

#define BT_RX 2
#define BT_TX 3
#define HOLD_THRESHOLD 800

Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT,
&Wire, OLED_RESET);
SoftwareSerial BT(BT_RX, BT_TX);

int centreX, centreY;
int letX = 56, letY = 24;

// AVAILABLE LETTERS
const char letters[] =
{'F','G','J','K','L','P','Q','R'};
const int totalLetters = 8;
int currentLetter = 0;

// HOLD TIMERS FOR LETTER CHANGE
unsigned long letterHoldStart = 0;
bool letterTriggered = false;
```

```
// BUTTON STATE
bool lastButtonState = HIGH;
unsigned long buttonPressTime = 0;
bool buttonHeld = false;

// STATUS
String statusText = "Ready";
String inputBuffer = "";

// SEND DATA TO SLAVE
void sendData() {
  String msg = "(";
  msg += letters[currentLetter];
  msg += ",";
  msg += String(letX);
  msg += ",";
  msg += String(letY);
  msg += ",0)";
  BT.println(msg);
  statusText = "Sent!";
}

// PARSE INCOMING PACKET FROM SLAVE
```

```
void parseIncoming(String line) {
  line.trim();
  if (!line.startsWith("(") || !line.endsWith(")")) {
    statusText = "Bad pkt";
    return;
  }
  line.remove(0, 1);
  line.remove(line.length() - 1);

  int p1 = line.indexOf(',');
  int p2 = line.indexOf(',', p1 + 1);
  int p3 = line.indexOf(',', p2 + 1);

  if (p1 < 0 || p2 < 0 || p3 < 0) {
    statusText = "Bad fmt";
    return;
  }

  char receivedChar = line.charAt(0);
  for (int i = 0; i < totalLetters; i++) {
    if (letters[i] == receivedChar) {
      currentLetter = i;
      break;
```

```
    }
  }


  letX = constrain(line.substring(p1 + 1,
p2).toInt(), 0, SCREEN_WIDTH  - 12);
  letY = constrain(line.substring(p2 + 1,
p3).toInt(), 0, SCREEN_HEIGHT - 16);


  statusText = "Recv:";
  statusText += receivedChar;
}


void setup() {
  Serial.begin(9600);
  BT.begin(9600);


  pinMode(JOY_SW, INPUT_PULLUP);  // button uses
internal pullup


  centreX = analogRead(JOY_X);
  centreY = analogRead(JOY_Y);


  if (!display.begin(SSD1306_SWITCHCAPVCC, 0x3C)) {
    if (!display.begin(SSD1306_SWITCHCAPVCC, 0x3D)) {
```

```
      Serial.println(F("OLED not found!"));
      while (true);
    }
  }

  display.clearDisplay();
  display.setTextSize(1);
  display.setTextColor(SSD1306_WHITE);
  display.setCursor(15, 20);
  display.println(F("Keep joystick still"));
  display.setCursor(25, 35);
  display.println(F("Calibrating..."));
  display.display();
  delay(1500);
  statusText = "Ready";
}

void loop() {
  int jx = analogRead(JOY_X) - centreX;
  int jy = analogRead(JOY_Y) - centreY;
  unsigned long now = millis();
  bool buttonState = digitalRead(JOY_SW);  // LOW
when pressed
```

```
  if (abs(jx) < DEADZONE) jx = 0;
  if (abs(jy) < DEADZONE) jy = 0;

  // DETECT BUTTON PRESS AND HOLD
  if (buttonState == LOW && lastButtonState == HIGH)
{
    // Button just pressed
    buttonPressTime = now;
    buttonHeld = false;
  }

  if (buttonState == LOW && (now - buttonPressTime >=
HOLD_THRESHOLD)) {
    // Button is being held
    buttonHeld = true;
  }

  if (buttonState == HIGH && lastButtonState == LOW)
{
    // Button just released
    if (!buttonHeld) {
      // Short press -> SEND
      sendData();
    }
```

```
    buttonHeld = false;
    letterTriggered = false;
  }

  lastButtonState = buttonState;

  // BUTTON HELD + JOYSTICK -> CHANGE LETTER
  if (buttonHeld) {
    statusText = "Btn held...";

    if ((jx > DEADZONE || jx < -DEADZONE || jy > DEADZONE || jy < -DEADZONE) && !letterTriggered) {
      if (jx > DEADZONE) {
        // Hold button + RIGHT -> next letter
        currentLetter = (currentLetter + 1) % totalLetters;
      } else if (jx < -DEADZONE) {
        // Hold button + LEFT -> previous letter
        currentLetter = (currentLetter - 1 + totalLetters) % totalLetters;
      } else if (jy > DEADZONE || jy < -DEADZONE) {
        // Hold button + UP or DOWN -> next letter (same as right)
        currentLetter = (currentLetter + 1) % totalLetters;
```

```
      }
      letterTriggered = true;
      statusText = "Letter:";
      statusText += letters[currentLetter];
    }

    // Reset trigger when joystick returns to centre
    if (jx == 0 && jy == 0) {
      letterTriggered = false;
    }

  } else {
    // NORMAL MOVEMENT — full freedom in all
directions
    if (jx > 0) letX += SPEED;
    if (jx < 0) letX -= SPEED;
    if (jy > 0) letY += SPEED;
    if (jy < 0) letY -= SPEED;
  }

  letX = constrain(letX, 0, SCREEN_WIDTH  - 12);
  letY = constrain(letY, 0, SCREEN_HEIGHT - 16);

  // DISPLAY
```

```
display.clearDisplay();

display.setTextSize(2);
display.setTextColor(SSD1306_WHITE);
display.setCursor(letX, letY);
display.print(letters[currentLetter]);

// Coordinates bottom left
display.setTextSize(1);
display.setCursor(0, 56);
display.print(F("X:")); display.print(letX);
display.print(F(" Y:")); display.print(letY);

// Status bottom right
display.setCursor(70, 56);
display.print(statusText);

display.display();
delay(30);

// BLUETOOTH RECEIVE
while (BT.available()) {
  char c = BT.read();
```

```
    if (c == '\n') {
      parseIncoming(inputBuffer);
      inputBuffer = "";
    } else if (c != '\r') {
      inputBuffer += c;
    }
  }
}
```

Code for Slave:

```
#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>
#include <SoftwareSerial.h>

#define SCREEN_WIDTH 128
#define SCREEN_HEIGHT 64
#define OLED_RESET -1

#define JOY_X A0
#define JOY_Y A1
#define SPEED 2
```

```
#define DEADZONE 80


#define BT_RX 2
#define BT_TX 3
#define HOLD_THRESHOLD 800


Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT,
&Wire, OLED_RESET);
SoftwareSerial BT(BT_RX, BT_TX);


int centreX, centreY;


// RECEIVED LETTER — this is what slave works with
char currentLetter = ' ';  // starts empty, fills
when master sends
int letX = 56, letY = 24;
bool letterReceived = false;  // nothing to show
until master sends


// AVAILABLE LETTERS FOR CYCLING
const char letters[] =
{'F','G','J','K','L','P','Q','R'};
const int totalLetters = 8;


// HOLD TIMERS
```

```
unsigned long downHoldStart  = 0;
unsigned long rightHoldStart = 0;
bool downTriggered  = false;
bool rightTriggered = false;

// STATUS
String statusText = "Waiting...";
String inputBuffer = "";

// SEND DATA BACK TO MASTER
void sendData() {
  if (!letterReceived) {
    statusText = "No letter!";
    return;
  }
  String msg = "(";
  msg += currentLetter;
  msg += ",";
  msg += String(letX);
  msg += ",";
  msg += String(letY);
  msg += ",0)";
  BT.println(msg);
```

```
  statusText = "Sent!";
}

// PARSE INCOMING PACKET FROM MASTER
void parseIncoming(String line) {
  line.trim();
  if (!line.startsWith("(") || !line.endsWith(")")) {
    statusText = "Bad packet";
    return;
  }
  line.remove(0, 1);
  line.remove(line.length() - 1);

  int p1 = line.indexOf(',');
  int p2 = line.indexOf(',', p1 + 1);
  int p3 = line.indexOf(',', p2 + 1);

  if (p1 < 0 || p2 < 0 || p3 < 0) {
    statusText = "Bad format";
    return;
  }

  // Take the letter and position from master
```

```
  currentLetter = line.charAt(0);
  letX = line.substring(p1 + 1, p2).toInt();
  letY = line.substring(p2 + 1, p3).toInt();

  letX = constrain(letX, 0, SCREEN_WIDTH  - 12);
  letY = constrain(letY, 0, SCREEN_HEIGHT - 16);

  letterReceived = true;
  statusText = "Got:";
  statusText += currentLetter;
}

// FIND NEXT LETTER IN ARRAY
int findLetterIndex(char c) {
  for (int i = 0; i < totalLetters; i++) {
    if (letters[i] == c) return i;
  }
  return 0;
}

void setup() {
  Serial.begin(9600);
  BT.begin(9600);
```

```
  centreX = analogRead(JOY_X);
  centreY = analogRead(JOY_Y);

  if (!display.begin(SSD1306_SWITCHCAPVCC, 0x3C)) {
    if (!display.begin(SSD1306_SWITCHCAPVCC, 0x3D)) {
      Serial.println(F("OLED not found!"));
      while (true);
    }
  }

  display.clearDisplay();
  display.setTextSize(1);
  display.setTextColor(SSD1306_WHITE);
  display.setCursor(15, 20);
  display.println(F("Keep joystick still"));
  display.setCursor(25, 35);
  display.println(F("Calibrating..."));
  display.display();
  delay(1500);
  statusText = "Waiting...";
}
```

```
void loop() {
  int jx = analogRead(JOY_X) - centreX;
  int jy = analogRead(JOY_Y) - centreY;
  unsigned long now = millis();

  if (abs(jx) < DEADZONE) jx = 0;
  if (abs(jy) < DEADZONE) jy = 0;

  // Only allow joystick control after letter is
received
  if (letterReceived) {

    // HOLD RIGHT -> CYCLE LETTER
    if (jx > DEADZONE) {
      if (rightHoldStart == 0) rightHoldStart = now;
      else if ((now - rightHoldStart >=
HOLD_THRESHOLD) && !rightTriggered) {
        int idx = findLetterIndex(currentLetter);
        currentLetter = letters[(idx + 1) %
totalLetters];
        rightTriggered = true;
        statusText = "Letter:";
        statusText += currentLetter;
      }
```

```
    } else {
      rightHoldStart = 0;
      rightTriggered = false;
    }

    // HOLD DOWN -> SEND BACK TO MASTER
    if (jy < -DEADZONE) {
      if (downHoldStart == 0) downHoldStart = now;
      else if ((now - downHoldStart >=
HOLD_THRESHOLD) && !downTriggered) {
        sendData();
        downTriggered = true;
      }
    } else {
      downHoldStart = 0;
      downTriggered = false;
    }

    // NORMAL MOVEMENT
    bool holdActive = (jx > DEADZONE) || (jy < -
DEADZONE);
    if (!holdActive) {
      if (jx > 0) letX += SPEED;
      if (jx < 0) letX -= SPEED;
```

```
      if (jy > 0) letY += SPEED;
      if (jy < 0) letY -= SPEED;
    }

    letX = constrain(letX, 0, SCREEN_WIDTH  - 12);
    letY = constrain(letY, 0, SCREEN_HEIGHT - 16);
  }

  // DISPLAY
  display.clearDisplay();

  if (!letterReceived) {
    // Show waiting message until master sends
something
    display.setTextSize(1);
    display.setTextColor(SSD1306_WHITE);
    display.setCursor(20, 28);
    display.print(F("Waiting for master"));
  } else {
    // Show the letter
    display.setTextSize(2);
    display.setTextColor(SSD1306_WHITE);
    display.setCursor(letX, letY);
    display.print(currentLetter);
```

```
  // Coordinates bottom left
  display.setTextSize(1);
  display.setCursor(0, 56);
  display.print(F("X:")); display.print(letX);
  display.print(F(" Y:")); display.print(letY);
}

// Status bottom right
display.setTextSize(1);
display.setCursor(70, 56);
display.print(statusText);

display.display();
delay(30);

// BLUETOOTH RECEIVE
while (BT.available()) {
  char c = BT.read();
  if (c == '\n') {
    parseIncoming(inputBuffer);
    inputBuffer = "";
  } else if (c != '\r') {
```

```
      inputBuffer += c;
    }
  }
```

10.References:

Adafruit Industries (2023) *Adafruit GFX Graphics Library*. Available at: https://learn.adafruit.com/adafruit-gfx-graphics-library (Accessed: 19 April 2026).

Banzi, M. and Shiloh, M. (2014) *Getting Started with Arduino*. 3rd edn. Sebastopol: O'Reilly Media.

Barr, M. and Massa, A. (2006) *Programming Embedded Systems: With C and GNU Development Tools*. 2nd edn. Sebastopol: O'Reilly Media.

Bluetooth SIG (2019) *Bluetooth Core Specification v5.2*. Available at: https://www.bluetooth.com/specifications/specs/core-specification/ (Accessed: 19 April 2026).

Buckley, J. (2013) *From Printout to Pixels: A History of Electronic Displays*. Cambridge: MIT Press.

Solomon Systech (2008) *SSD1306 Datasheet: 128 x 64 Dot Matrix OLED/PLED Segment/Common Driver with Controller*. Available at: https://cdn-shop.adafruit.com/datasheets/SSD1306.pdf (Accessed: 19 April 2026).

Microchip Technology (2021) *ATmega328P Datasheet*. Available at: https://ww1.microchip.com/downloads/en/DeviceDoc/Atmel-7810-Automotive-Microcontrollers-ATmega328P_Datasheet.pdf (Accessed: 3 May 2026).

Texas Instruments (2015) *CC2541 Bluetooth Low Energy System-on-Chip Datasheet*. Available at: https://www.ti.com/lit/ds/symlink/cc2541.pdf (Accessed: 3 May 2026).

UK Government (2013) *The Waste Electrical and Electronic Equipment Regulations 2013*. Available at: https://www.legislation.gov.uk/uksi/2013/3113/contents (Accessed: 3 May 2026).

Wavesen (2011) *HC-05 Bluetooth Serial Communication Module Datasheet*. Available at: https://www.electronicaestudio.com/docs/istd016A.pdf (Accessed: 3 May 2026).

Williams, E., Ayres, R. and Heller, M. (2002) 'The 1.7 kilogram microchip: energy and material use in the production of semiconductor devices', *Environmental Science & Technology*, 36(24), pp. 5504–5510